

1. INSTALLING & USING VIM (Essential Editor)

Installation

```
sudo apt-get install vim
```

Basic Vim Workflow

Action	Command	Description
Enter Insert Mode	i	Start typing
Enter Append Mode	a	Append at cursor
Exit Insert Mode	Esc	Back to command mode
Save	:w	Write (save) file
Save & Quit	:wq	Write & quit
Quit (discard changes)	:q!	Quit without saving

Navigation (Command Mode)

Move	Command	Description
Word Forward	w	Next word
Word Backward	b	Previous word
Line Start	0	Beginning of line
Line End	\$	End of line
File End	G	Last line
File Start	gg	First line
Arrows	↑ ↓ ← →	Normal movement

Editing Commands

Action	Command	Description
Delete Line	dd	Delete current line
Delete All	ggdG	Delete entire file
Copy Line	yy	Yank (copy) line

Action	Command	Description
Paste	p	Paste after cursor
Select Block	V + arrows	Visual block select
Copy Selection	V + y	Yank selection
Delete Selection	V + d	Delete selection

Vim Settings (Permanent)

Create ~/.vimrc file:

```
vim ~/.vimrc
```

Add these lines:

```
set number          " Show line numbers
syntax on           " Syntax highlighting
set tabstop=2       " 2 spaces per tab
set autoindent      " Auto indent
```

Verify vimrc: echo \$MYVIMRC

Reload: Restart terminal

2. BASIC SCRIPT STRUCTURE

Hello World Script

```
#!/bin/bash          # Shebang (interpreter)
# Comment
echo "Hello World"
```

Make Executable

```
chmod 755 script.sh  # Owner: rwx | Group/Others: rx
./script.sh          # Execute
```

Permission Numbers

Number	Read	Write	Execute	Meaning
7	✓	✓	✓	Full
6	✓	✓	✗	Read+Write
5	✓	✗	✓	Read+Execu

Number	Read	Write	Execute	Meaning
				te
4	✓	X	X	Read only
3	X	✓	✓	Write+Execute
2	X	✓	X	Write only
1	X	X	✓	Execute only
0	X	X	X	None

Order: Owner | Group | Others

3. VARIABLES

Declaring Variables

```
my_name="Derick"      # No spaces around =
num1=5
```

Rules: Start with letter/underscore, then letters/numbers/underscores

Constants

```
declare -r num1=5      # Read-only
```

Accessing Variables

```
echo $my_name          # $var_name
```

Arithmetic Variables

```
num3=$((num1 + num2))  # Double brackets for math
```

All Math Operators:

Operation	Symbol	Example
Add	+	<code>\$((5 + 3))</code>
Subtract	-	<code>\$((5 - 3))</code>
Multiply	*	<code>\$((5 * 3))</code>
Divide	/	<code>\$((5 / 3))</code>
Modulus	%	<code>\$((5 % 3))</code>

Operation	Symbol	Example
Exponent	**	<code>\$((2 ** 3))</code>

Assignment Shorthand

```
i+=2      # i = i + 2
i-=2      # i = i - 2
i*=2      # i = i * 2
i/=2      # i = i / 2
```

Increment/Decrement

```
i++      # Post: print then increment
++i      # Pre: increment then print
i--      # Post: print then decrement
--i      # Pre: decrement then print
```

Example:

```
rand=9
echo $rand      # 9
echo $((rand++)) # 9 (then rand=10)
echo $((++rand)) # 11
```

Floating Point (Using Python)

```
num7=1.2
num8=3.4
num9=$(python3 -c "print(num7 + num8)") # 4.6
```

4. OUTPUT METHODS

Echo

```
echo "Hello $name"
echo -e "Line1\nLine2" # -e enables escapes
```

Printf

```
printf "Average: %.3f\n" $avg
```

Here Document (Multi-line)

```
cat << EOF
Line 1
```

Line 2
EOF

5. FUNCTIONS

Basic Function

```
get_date() {  
    date  
    return          # Exit status 0-255  
}  
get_date           # Call function
```

Global vs Local Variables

```
name="Derick"      # Global  
  
demo_local() {  
    local name="local" # Local (only in function)  
    echo $name         # Prints: local  
}  
demo_local  
echo $name            # Prints: Derick
```

Functions with Parameters

```
get_sum() {  
    local num3=$1      # First arg  
    local num4=$2      # Second arg  
    sum=$((num3 + num4))  
    echo $sum          # Return value  
}  
  
num1=5  
num2=6  
result=$(get_sum $num1 $num2) # result=11  
echo "The sum is $result"
```

Args: \$1, \$2, \$3, ... \$9, \${10}, \${11}

6. INPUT (Read)

Basic Input

```
read -p "What is your name? " name
echo "Hello $name"
```

Multiple Inputs

```
read -p "Enter two numbers: " num1 num2
echo $((num1 + num2))
```

Hidden Input

```
read -s -p "Secret code: " secret
```

Custom Separator (IFS)

```
old_ifs=$IFS
IFS=,
read -p "Numbers (comma sep): " num1 num2
num1=${num1// /}    # Remove spaces
num2=${num2// /}
IFS=$old_ifs
```

7. CONDITIONALS

If Statement

```
if [ $age -ge 16 ]; then
    echo "You can drive"
elif [ $age -eq 15 ]; then
    echo "Next year"
else
    echo "Can't drive"
fi
```

Test Operators

Type	Symbol	Example	Meaning
Numeric	-eq	[5 -eq 5]	Equal
	-ne	[5 -ne 3]	Not equal
	-gt	[5 -gt 3]	Greater than
	-ge	[5 -ge 5]	≥

Type	Symbol	Example	Meaning
	-lt	[3 -lt 5]	Less than
	-le	[3 -le 5]	≤

String	Symbol	Example	Meaning
Empty	-z	[-z "\$str"]	Zero length
Not Empty	-n	[-n "\$str"]	Not zero
Equal	=	["\$str1" = "\$str2"]	Equal
Not Equal	!=	["\$str1" != "\$str2"]	Not equal

File	Symbol	Example	Meaning
Exists	-e	[-e "file"]	File exists
Directory	-d	[-d "dir"]	Is directory
Readable	-r	[-r "file"]	Readable
Writable	-w	[-w "file"]	Writable
Executable	-x	[-x "file"]	Executable

Logical Operators

Operator	Symbol	Example
AND	&&	[1 -gt 0] && [1 -lt 10]
OR		[1 -eq 10] [1 -eq 1]
NOT	!	[! -z "\$str"]

In Brackets:

```
if [[ $num -gt 0 && $num -lt 11 ]]; then
    echo "Between 1-10"
fi
```

Even/Odd Check

```
if [ $(num % 2) -eq 0 ]; then
    echo "Even"
fi
```

File Commands

```
touch sample.txt          # Create file
mkdir sample_dir          # Create dir
rm sample.txt             # Delete file
rmdir sample_dir          # Delete dir
```

Example: Create if doesn't exist

```
[ ! -d "sample_dir" ] && mkdir sample_dir
```

Regex Validation

```
read -p "Enter date (8 digits): " date
pattern="^[0-9]{8}$"
if [[ $date =~ $pattern ]]; then
    echo "Valid date"
else
    echo "Invalid"
fi
```

Case Statement

```
read -p "Age: " age
case $age in
    [0-4])
        echo "Too young for school" ;;
    5)
        echo "Kindergarten" ;;
    [6-9]|[10-18])
        grade=$((age - 5))
        echo "Go to grade $grade" ;;
    *)
        echo "Too old for school" ;;
esac
```

Ternary Operator

```
age=18
can_vote=$(( [ $age -ge 18 ] && echo 1 || echo 0 )
echo $can_vote # 1
```

8. PARAMETER EXPANSION (Strings)

Syntax: `${variable/pattern/replacement}`

Use Case	Example	Output
Add text	<code>name="Derick"; echo \${name}s toy</code>	Dericks toy
Replace	<code>str="dog house"; echo \${str/dog/cat}</code>	cat house
Remove spaces	<code>num1=" 23 "; echo \${num1// /}</code>	23
Length	<code>str="hello"; echo \${#str}</code>	5
Substring	<code>\${str:2}</code>	llo (from index 2)
Range	<code>\${str:2:3}</code>	llo
After pattern	<code>\${str#a* }</code>	string (after "a ")

Default Value:

```
echo ${name:-Derick}    # Derick if name empty
name=${name:-Derick}    # Set default
```

9. LOOPS

While Loop

```
num=1
while [ $num -le 10 ]; do
    echo $num
    ((num++))
done
```

Until Loop (opposite of while)

```
num=1
until [ $num -gt 10 ]; do
    echo $num
    ((num++))
done
```

Continue & Break

```
num=1
while [ $num -le 20 ]; do
    if [ $((num % 2)) -eq 0 ]; then
        ((num++))
        continue    # Skip even numbers
    fi
    if [ $num -ge 15 ]; then
        break        # Exit loop
    fi
    echo $num
    ((num++))
done
```

Read File Loop

```
while read avg rbi hr; do
    printf "Avg: %s\nRBI: %s\nHR: %s\n\n" "$avg" "$rbi" "$hr"
done < barry_bonds.txt
```

For Loop (C-style)

```
for ((i=0; i<=10; i++)); do
    echo $i
done
```

For Loop (Range)

```
for i in {0..10}; do
    echo $i
done
```

10. ARRAYS (1D Only)

Create Array

```
favorite_nums=(3.14 27 8 57721)    # No commas
```

Access

Access	Example	Output
Element	<code>\$</code> <code>{favorite_nums[0]}</code>	3.14
All	<code>\$</code>	All elements

Access	Example	Output
	<code>{favorite_nums[*]}</code>	
Length	<code>\$ {#favorite_nums[@] }</code>	4
Element Length	<code>\$ {#favorite_nums[2] }</code>	Length of 3rd

Add Elements

```
favorite_nums[4]=42      # Add single
favorite_nums+=(1 7)     # Add multiple
```

Loop Through Array

```
for num in "${favorite_nums[@]}"; do
    echo $num
done
```

Sort Array

```
sorted_nums=($(for i in "${favorite_nums[@]}"; do echo $i; done |
sort))
```

Delete

```
unset favorite_nums[1]    # Delete element
unset favorite_nums       # Delete array
```

Print Indexes:

```
for i in "${!favorite_nums[@]}"; do
    echo "Index $i: ${favorite_nums[$i]}"
done
```

11. POSITIONAL PARAMETERS

\$0: Script name/path
\$1-\$9: Arguments 1-9
\${10}: Argument 10+

Example: Sum Arguments

```
echo "First arg: $1"
```

```
sum=0
while [ $# -gt 0 ]; do      # $# = number of args
    sum=$((sum + $1))
    shift                  # Move args left
done
echo "Sum: $sum"
```

Run: ./script.sh 1 2 3 4 5 **Output:** First arg: 1, Sum: 15

12. CHEAT SHEET COMMANDS

Task	Command
List files	ls -la
Clear screen	clear
Current dir	pwd
Run script	./script.sh
Edit	vim script.sh
Delete all in vim	ggdG
Copy line	yy
Paste	p

COMPLETE PRACTICE SCRIPT

Save as practice.sh, chmod 755 practice.sh, ./practice.sh

```
#!/bin/bash
```

```
# Variables & Math
```

```
name="Derick"
```

```
age=25
```

```
pi=3.14
```

```
# Function
```

```
greet() { echo "Hello $1!"; }
```

```
# Input
read -p "Your name: " user

# Array
colors=("red" "green" "blue")

# Loop
for color in "${colors[@]}; do
    echo "Color: $color"
done

# Conditional
if [ $age -ge 18 ]; then
    echo "Adult"
else
    echo "Minor"
fi

# Output
greet $user
echo "PI: $pi"
```